

Laboranweisung 10: Sequenzielle Logik, Takt & das D-Flip-Flop

Heute stattdessen wir unseren Simulator mit einem Gedächtnis aus, führen eine künstliche Zeitachse (Clock) ein und bewältigen das verbotene Thema der letzten Woche: Die Ringverkabelung (Zyklen). Am Ende werden Sie Ihren ersten funktionierenden 1-Bit Zähler gelötet haben.

Phase 1: Die Klasse DFlipFlop (Das Gedächtnis)

Das elementarste Speicherbauteil fungiert als Firewall in unserem Graphen und zerschneidet Endlosschleifen.

1. Erstellen Sie die Dateien DFlipFlop.h und DFlipFlop.cpp.
2. Vererbung & Konstruktor: Erben Sie von Gate und dimensionieren Sie m_inputs auf exakt 1.
3. Legen Sie eine private Variable `bool m_storedState = false;` an. Das ist Ihr Tresor.
4. Überschreiben Sie `evaluate()`. Achtung, Firewall: Rufen Sie hier nicht `evaluate()` auf den Inputs auf! Setzen Sie einfach `m_output = m_storedState;` (und ggf. `m_isCalculated = true;`).
5. Erstellen Sie `void onClockTick()`. Diese Methode simuliert die Taktflanke: Lesen Sie den Wert an Pin 0 mit `getOutput()` aus und speichern Sie ihn in `m_storedState`.

Phase 2: Die 2-Phasen-Schleife & DFS-Korrektur

Lesen (Kombinatorik) und Schreiben (Flip-Flops) müssen strikt voneinander getrennt werden.

1. Legen Sie in der `main.cpp` zwei Vektoren an: `std::vector<std::shared_ptr<Gate>> allGates` (für den Cache-Reset) und `std::vector<std::shared_ptr<DFlipFlop>> allFlipFlops` (für das Taktsignal).
2. Bauen Sie eine `for`-Schleife für 10 Taktzyklen.
3. Schritt A (Amnesie): Iterieren Sie über `allGates` und rufen Sie `reset()` auf.
4. Schritt B (Lese-Phase): Wichtige architektonische Erkenntnis: Da das D-Flip-Flop rekursive Aufrufe blockiert, würde die Logik davor niemals berechnet werden! Sie müssen `evaluate()` zwingend auf dem Gatter aufrufen, das vor dem D-Flip-Flop hängt (oder auf allen Kombinatorik-Endpunkten).
5. Schritt C (Ausgabe): Drucken Sie den Wert des D-Flip-Flops in die Konsole.
6. Schritt D (Schreib-Phase): Iterieren Sie über `allFlipFlops` und rufen Sie `onClockTick()` auf. Die Flip-Flops speichern nun die stabilen Signale.

Phase 3: Der 1-Bit Zähler (Die Feuerprobe)

Wir bauen absichtlich einen Ring, der ohne DFF sofort zum Absturz führen würde.

1. Instanzieren Sie ein DFlipFlop und ein NotGate.
2. Die Ringverkabelung:
 - Verbinden Sie den Eingang des NotGate mit dem Ausgang des DFlipFlop.
 - Verbinden Sie den Eingang des DFlipFlop mit dem Ausgang des NotGate.
3. Rufen Sie in Schritt B Ihrer Schleife (Lese-Phase) evaluate() auf dem NotGate auf.
4. Starten Sie das Programm. Gibt Ihre Konsole abwechselnd 0, 1, 0, 1 aus, ohne abzustürzen? Herzlichen Glückwunsch!

Fehler-Checkliste

- Stack Overflow: Haben Sie in der evaluate()-Methode des DFlipFlop versehentlich den Input rekursiv ausgewertet?
- Der Output ändert sich nie (bleibt auf 0): Haben Sie vergessen, in der Schleife evaluate() auf dem NOT-Gatter aufzurufen? Wenn das NOT-Gatter nicht rechnet, hat das DFF bei der Flanke immer nur den Wert false vorliegen!
- Kompilierungsfehler: Ein normales Gate kennt die Methode onClockTick() nicht. Nutzen Sie den separaten allFlipFlops-Vektor (Typ: std::shared_ptr<DFlipFlop>).

Zusatzaufgaben (Für Schnelle)

Zusatzaufgabe 1: D-Flip-Flop mit Enable-Pin (DFFE)

Erweitern Sie Ihr DFlipFlop auf 2 Eingänge. Pin 1 wird das Enable-Signal (EN). Ändern Sie onClockTick() so, dass der Wert von Pin 0 nur in m_storedState übernommen wird, wenn der Zustand von Pin 1 auf true steht.

Zusatzaufgabe 2: Der 2-Bit Zähler

Bauen Sie einen Zähler, der von 0 bis 3 zählt (00, 01, 10, 11 → 00).

- Sie benötigen zwei D-Flip-Flops (Bit 0 und Bit 1).
- Bit 0 toggelt bei jedem Takt (wie in Phase 3).
- Bit 1 toggelt nur, wenn Bit 0 aktuell 1 ist (Nutzen Sie ein XOR-Gatter aus Bit 0 und Bit 1, das wieder in Bit 1 geführt wird).